Functions, errors, and tests

Previously

We wrote a function to compute the sample variance:

```
1 my_var <- function(x){
2   sum((x - mean(x))^2)/(length(x) - 1)
3 }
```

What assumptions is this function making?

Potential sources of problems

```r
my_var <- function(x){
  sum((x - mean(x))^2)/(length(x) - 1)
}

my_var(2)
```

```
[1] NaN
```

```r
my_var(c('a', 'b'))
```

```
Error in x - mean(x): non-numeric argument to binary operator
```

```r
my_var(c(1, 2, NA, 3))
```

```
[1] NA
```

Modifying our function

If our function cannot run (e.g., if there is an issue with the provided input), then our function should throw an **error**.

```r
my_var <- function(x){
  if(length(x) < 2){
    stop("Input must have at least two values.")
  }

  sum((x - mean(x))^2)/(length(x) - 1)
}

my_var(2)
```

```
Error in my_var(2): Input must have at least two values.
```

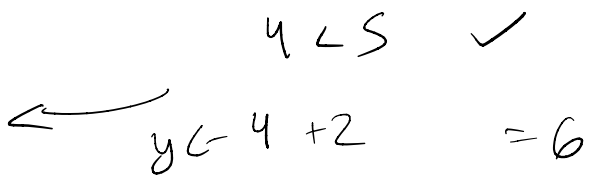
```r
my_var(c(1, 2))
```

```
[1] 0.5
```

Choices

What will the following code return?

```
1 y <- 4
2
3 if(y < 5){
4   y <- y + 2
5 }
6
7 y
```



6

Choices

What will the following code return?

```r
y <- 4

if(y < 5){
  y <- y + 2
}

y
```

```
[1] 6
```

Choices

What will the following code return?

```
y <- 7

if(y < 5){
  y <- y + 2
}

y
```

Choices

What will the following code return?

```
y <- 7

if(y < 5){
  y <- y + 2
}

y
```

```
[1] 7
```

The code within an **if** statement is run *only* if the specified condition is TRUE

Choices

What will the following code return?

```r
y <- 7

if(y < 5){
  y <- y + 2
} else {
  y <- y - 3
}

y
```

Choices

What will the following code return?

```
y <- 7

if(y < 5){
  y <- y + 2
} else {
  y <- y - 3
}

y
```

```
[1] 4
```

If the condition for an `if` statement is `FALSE`, then the code inside the (optional) `else` statement will run.

Choices

We can use `else if` to combine multiple `if`/`else` blocks:

```
y <- 7

if(y < 5){
  y <- y + 2
} else if(y > 5){
  y <- y - 3
} else {
  print("y is 5!")
}

y
```

```
[1] 4
```

Choices

We can use `else if` to combine multiple `if`/`else` blocks:

```
y <- 5

if(y < 5){
  y <- y + 2
} else if(y > 5){
  y <- y - 3
} else {
  print("y is 5!")
}
```

```
[1] "y is 5!"
```

```
y
```

```
[1] 5
```

Choices

What will the following code produce?

```
y <- 4

if(y < 5){
  y <- y + 2
}

if(y > 5){
  y <- y - 3
}

y
```

Choices

What will the following code produce?

```
y <- 4

if(y < 5){
  y <- y + 2
}

if(y > 5){
  y <- y - 3
}

y
```

```
[1] 3
```

Choices

What will the following code produce?

```
y <- 4

if(y < 5){
  y <- y + 2
} else if(y > 5){
  y <- y - 3
}

y
```

Choices

What will the following code produce?

```
y <- 4

if(y < 5){
  y <- y + 2
} else if(y > 5){
  y <- y - 3
}

y
```

```
[1] 6
```

Summary so far

```
my_var <- function(x){
  if(length(x) < 2){
    stop("Input must have at least two values.")
  }

  sum((x - mean(x))^2)/(length(x) - 1)
}
```

- Use an `if` statement to check if the provided input does not match our expectations
- Use `stop` to throw an error (with a helpful message) and exit the function

Exiting a function

R supports **implicit** returns:

```
1 my_var <- function(x){
2   if(length(x) < 2){
3     stop("Input must have at least two values.")
4   }
5
6   sum((x - mean(x))^2)/(length(x) - 1)
7 }
8
9 my_var(c(1, 2))
```

```
[1] 0.5
```

Exiting a function

These can also be made **explicit**:

```
my_var <- function(x){
  if(length(x) < 2){
    stop("Input must have at least two values.")
  }

  return(sum((x - mean(x))^2)/(length(x) - 1))
}

my_var(c(1, 2))
```

```
[1] 0.5
```

Exiting a function

What do you think the following code will return?

```
1 g <- function(x){
2   x + 1
3   return(x)
4 }
5
6 g(5)
```

<- won't exit on line 2, because we have more code below in function

explicit return : return x

g(5) -> 5

if

```
g <- function(x){
    x <- x+1
    return(x)
}
g(5)
```

-> 6

Exiting a function

What do you think the following code will return?

```r
g <- function(x){
  x + 1
  return(x)
}

g(5)
```

```
[1] 5
```

Exiting a function

What do you think the following code will return?

```
g <- function(x){
  return(x + 1)
  x <- x + 2
  x
}

g(5)
```

Exiting a function

What do you think the following code will return?

```
1 g <- function(x){
2   return(x + 1)
3   x <- x + 2
4   x
5 }
6
7 g(5)
```

```
[1] 6
```

once return is executed, we exit the function

never gets executed

Exiting a function

What do you think the following code will return?

```
g <- function(x){
  stop("Error!")
  return(x + 1)
}

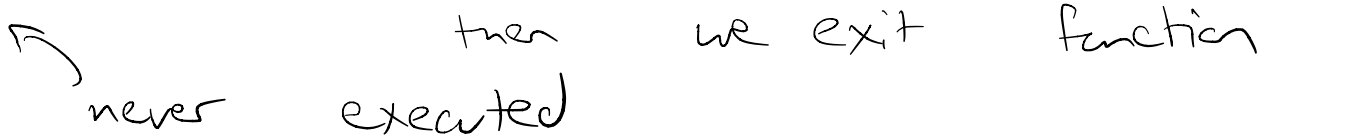
g(5)
```

Exiting a function

What do you think the following code will return?

```
1 g <- function(x){
2   stop("Error!")
3   return(x + 1)
4 }
5
6 g(5)
```

```
Error in g(5): Error!
```



Exiting a function

What do you think the following code will return?

```
g <- function(x){
  return(x + 1)
  stop("Error!")
}

g(5)
```

Exiting a function

What do you think the following code will return?

```r
g <- function(x){
  return(x + 1)
  stop("Error!")
}

g(5)
```

```
[1] 6
```

Exiting a function:

- if we execute stop or return, exit function immediately
- if all code in function executes (don't execute any stop, exit, etc.) then R does implicit return